

## 1. Introducción

El proyecto de Gestión de Imprenta nace como una propuesta para modernizar la forma en que se administran las solicitudes de impresión. Específicamente dentro de un entorno escolar. En muchos casos, este tipo de procesos se sigue realizando de manera manual, usando cuadernos, hojas o registros físicos. Esto puede funcionar al principio, pero con el tiempo se vuelve poco práctico, especialmente cuando se necesita buscar una solicitud antigua, revisar quién pidió una impresión, cuándo fue realizada, cuántas copias se solicitaron o si el trabajo ya fue retirado.

Un ejemplo claro de este problema ocurre cuando se necesita consultar una impresión solicitada hace varios meses. Si toda la información está anotada en cuadernos, la búsqueda puede volverse lenta, desordenada y dependiente de que esos registros físicos estén completos y disponibles. Además, al manejar la información manualmente, aumenta la posibilidad de errores, pérdidas de datos o falta de seguimiento sobre el estado real de cada solicitud.

El sistema propone una solución digital que permite registrar, consultar y organizar la información de forma más rápida y segura. La idea principal es que cada solicitud de impresión quede guardada en el sistema junto con datos importantes como la cantidad de copias, estado, fecha de solicitud y notas adicionales.

Además, al consultar una impresión el sistema mostrará información relacionada al profesor, curso y asignatura. De esta forma cuando una impresión queda registrada se podrá saber el correo del profesor para tener un flujo de trabajo más rápido.

También se incorpora de forma complementaria y opcional el uso de inventario, cola de impresión, control de calidad y retiros. Estos módulos ayudan a mantener una constancia o hacer chequeos rápidos desde el mismo pc.

Este proyecto se trabajó con microservicios, separando el sistema en varias partes para que cada una tuviera una función específica. También se agregó un API Gateway para entrar a los servicios desde un solo puerto, autenticación con Login por token para seguridad, se le implementó Swagger, pruebas unitarias y bases de datos separadas para cada microservicio.

## 2. Contexto del proyecto y propuesta de solución

El proyecto está enfocado específicamente para un colegio en el cual se manejan miles de copias al mes, donde distintos profesores pueden solicitar impresiones para sus cursos y asignaturas. En este tipo de entorno, es importante mantener un registro claro de cada solicitud, ya que se debe saber quién pidió la impresión, para qué curso era, a qué asignatura corresponde, cuántas copias se solicitaron, si ya fue retirado o no y anotar algo adicional como extra.

La propuesta de solución consiste en crear un sistema backend basado en microservicios que permita manejar esta información de forma digital. En lugar de depender de registros escritos en cuadernos, el sistema permite guardar las solicitudes en una base de datos y consultarlas cuando sea necesario. Esto ayuda a reducir el desorden, evitar pérdidas de información y facilitar el seguimiento de cada impresión.

El servicio principal del sistema es el microservicio de impresiones, ya que desde ahí se registran y consultan las solicitudes. Este servicio se apoya en otros microservicios, como profesores, cursos y asignaturas, para mostrar información más completa al momento de consultar una impresión. De esta manera, el sistema no solo muestra el registro de la impresión, sino también los datos relacionados que ayudan a entender mejor la solicitud.

Además, el proyecto cuenta con servicios complementarios como inventario, cola de impresión, control de calidad y retiros. Estos servirán para mantener constancia desde el mismo computador.

**Inventario:** Permitirá anotar materiales y sumarle o descontarle cantidades.

**Cola:** Permitirá ver impresiones si están en estado “Urgente” “En cola” o “Lista”

**Calidad:** Permitirá asociar una impresión y anotar si salió con defectos o algún problema

**Retiros:** Permitirá asociarla a una impresión y saber si esta ya fue retirada o está pendiente

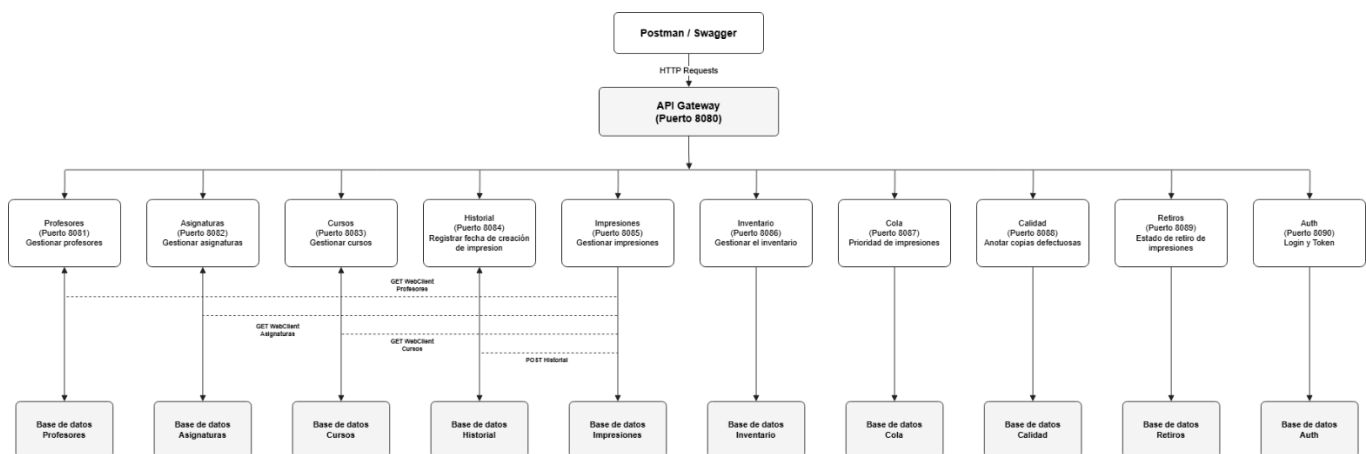
### 3. Arquitectura del sistema

El sistema fue desarrollado utilizando microservicios. Esto quiere decir que el proyecto se separó en varios servicios independientes, donde cada uno cumple una función específica dentro del sistema. Por ejemplo, existen servicios para profesores, asignaturas, cursos, impresiones, historial, inventario, cola, calidad, retiros y autenticación.

Para evitar tener que acceder a cada microservicio desde un puerto distinto, se implementó el **API Gateway**. Este funciona como una entrada principal al sistema, permitiendo que las solicitudes se realicen desde un solo puerto, en este caso el puerto 8080.

Cada microservicio tiene su propia base de datos, lo que permite separar la información y evitar que, si es que se cae un microservicio, el proyecto no funcione por completo. Esto también ayuda a mantener el proyecto más ordenado.

Dentro de esta arquitectura, el microservicio de impresiones es el más importante, ya que se comunica con otros servicios para obtener información que utilizara como profesor, curso y asignatura. Gracias a esto, al consultar una impresión, el sistema puede entregar una respuesta más completa.



En el diagrama se puede ver que Postman o Swagger realizan solicitudes hacia el API Gateway. Luego, el Gateway redirige la solicitud al microservicio correspondiente. También se muestra que cada microservicio cuenta con su propia base de datos y que el servicio de impresiones se comunica con otros servicios para reunir la información necesaria de cada solicitud.

[https://drive.google.com/file/d/1HibKLFC4OOodaLUIYm\\_hMM\\_UO\\_\\_\\_6Bg/view?usp=sharing](https://drive.google.com/file/d/1HibKLFC4OOodaLUIYm_hMM_UO___6Bg/view?usp=sharing)

(Imagen en más calidad)

#### 4. Microservicios implementados

El sistema está compuesto por 10 microservicios principales, sin contar el API Gateway. Cada microservicio cumple una función específica dentro del sistema, lo que permite separar responsabilidades y sea más fácil de mantener.

Además de los microservicios principales, se utiliza un API Gateway en el puerto 8080, el cual funciona como punto de entrada centralizado para acceder a los servicios desde una sola ruta, es decir no hay que cambiar el puerto a cada rato.

| Servicio            | Puerto | Función                                                                                |
|---------------------|--------|----------------------------------------------------------------------------------------|
| api-Gateway         | 8080   | Centraliza las rutas del sistema                                                       |
| service-profesores  | 8081   | Gestiona la información de los profesores                                              |
| service-asignaturas | 8082   | Gestiona las asignaturas                                                               |
| service-cursos      | 8083   | Gestiona los cursos                                                                    |
| service-historial   | 8084   | Registra acciones relacionadas con las impresiones                                     |
| service-impresiones | 8085   | Gestiona las solicitudes de impresión                                                  |
| service-inventario  | 8086   | Permite registrar y actualizar materiales                                              |
| service-cola        | 8087   | Permite ver el estado de una impresión asociada y ver si es urgente o ya fue entregada |
| service-calidad     | 8088   | Permite anotar problemas o copias defectuosas                                          |
| service-retiros     | 8089   | Permite registrar si una impresión fue retirada o sigue pendiente                      |
| service-Auth        | 8090   | Gestiona registro, Login y generación de token JWT                                     |

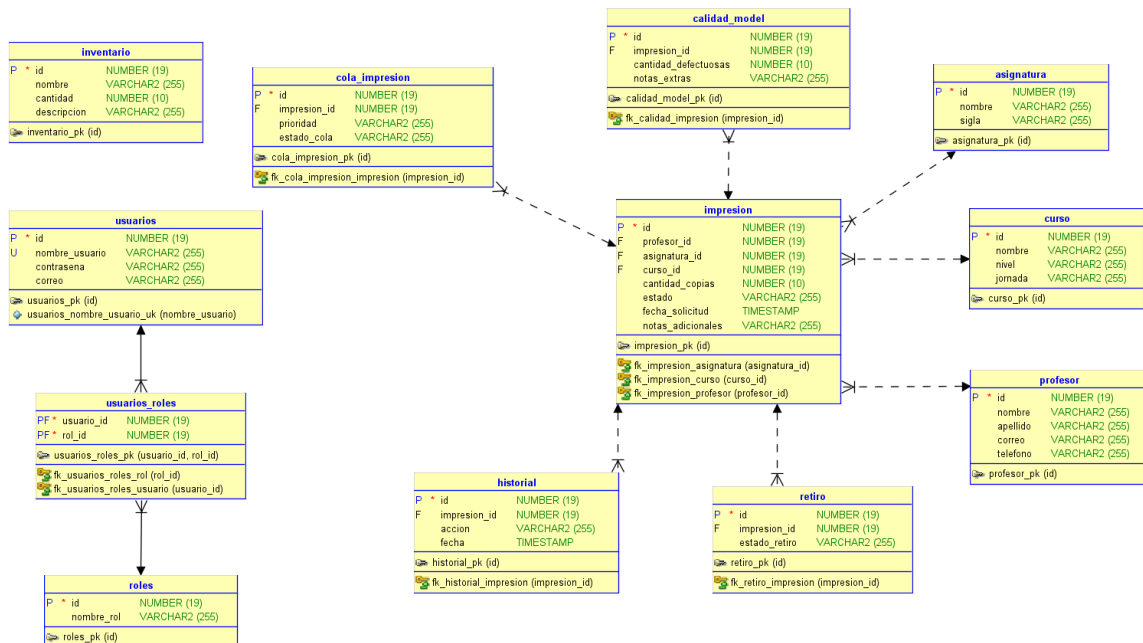
De esta forma, el proyecto mantiene una separación clara entre las distintas partes del sistema. Por ejemplo, si se quiere editar la información de un profesor, se hará de forma independiente sin cambiar todo el sistema por completo.

## 5. Modelo de base de datos

El sistema trabaja con bases de datos separadas para cada microservicio. Esto quiere decir que cada servicio administra su propia información y no depende de una única base de datos central. Por ejemplo, el microservicio de profesores utiliza su propia base, el de cursos otra, el de impresiones otra.

La tabla principal es **impresión**, ya que se anota una solicitud y se asocia a un profesor. Esto permite saber quién la solicitó, a qué curso corresponde y a qué asignatura pertenece, también mostrar información adicional, como el correo del profesor.

También existen tablas complementarias como: Historial, Cola, Calidad y Retiro. Estas permiten registrar acciones, estado, anotar problemas de calidad o si la impresión ya fue retirada. Por otro lado, Inventario y Usuarios funcionan como tablas independientes, ya que cumplen funciones separadas dentro del sistema.



El modelo representa las relaciones lógicas entre las entidades principales del sistema.

En la realidad, cada microservicio mantiene su propia base de datos independiente, pero el diagrama permite ver como la información se relaciona con una solicitud de impresión.

## 6. API Gateway

El API Gateway cumple la función de que el proyecto este centralizado. En vez de entrar manualmente a cada microservicio por su propio puerto, todas las solicitudes pasaran primero por el Gateway usando el puerto 8080.

Esto hace que el uso del sistema sea más ordenado, ya que desde Postman o Swagger se puede trabajar con rutas como:

/profesores, /impresiones, /inventario o /cola.

Sin tener que recordar el puerto individual de cada microservicio. Luego el Gateway se encarga de redirigir la solicitud al servicio que corresponde.

Por ejemplo, si se realiza una petición a:

<http://localhost:8080/profesores>

El Gateway redirige esa solicitud al microservicio de profesores, la cual funciona en el puerto **8081**. Lo mismo ocurre con impresiones, cursos, asignaturas, historial, inventario, cola, calidad, retiros y autenticación.

Además, se le agregó autenticación para seguridad. Esto permite que algunas rutas queden protegidas y solo puedan ser utilizadas si el usuario primero inicia sesión y obtiene un token.

Las rutas de autenticación: /auth/register y /auth/login, quedan libres para poder registrarse e iniciar sesión, mientras que las demás rutas necesitan el token para funcionar.

De esta forma, el API Gateway no solo sirve para centralizar las rutas, sino también para controlar el acceso al sistema y trabajar de forma más fácil y rápida entre los microservicios.

## 7. Autenticación y seguridad JWT

Para agregar seguridad al sistema se implementó un microservicio de autenticación llamado **service-auth**. Este servicio permite registrar usuarios e iniciar sesión. La idea principal es que no cualquier persona pueda entrar directamente a las rutas del sistema sin antes autenticarse.

La idea de usar JWT es que el usuario primero debe iniciar sesión con su nombre de usuario y contraseña. Si las credenciales son correctas, el sistema devuelve un token. Luego, ese token se utiliza en Postman dentro de la pestaña

**Authorization**, luego se selecciona la opción, **Bearer Token**. De esta forma, el usuario puede acceder a las rutas protegidas.

En este proyecto, las rutas de autenticación quedan libres para poder registrarse e iniciar sesión:

POST <http://localhost:8080/auth/register>

POST <http://localhost:8080/auth/login>

Las demás rutas del sistema quedan protegidas. Por ejemplo, si se intenta acceder a una ruta como:

GET <http://localhost:8080/profesores>

Para que Swagger pudiera funcionar correctamente desde el puerto 8080. No se le implemento Auth al Swagger. Esto permite visualizar y seleccionar los microservicios desde Swagger sin necesidad de token. Sin embargo, las rutas funcionales del sistema siguen protegidas, por lo que al probarlas desde Postman se debe iniciar sesión y enviar el token JWT como Bearer Token.

También se utilizó Spring Security para manejar la seguridad del microservicio de autenticación. Las contraseñas no se guardan como texto normal, sino que quedan encriptadas. Esto permitirá que, aunque la información quede en la base de datos, la contraseña real del usuario no sea visible directamente.

El API Gateway también cumple un rol importante en esta parte, ya que funciona como filtro de seguridad. Antes de dejar pasar una solicitud hacia los microservicios protegidos, revisa si la petición trae un token válido. Si el token no existe o no es correcto, la solicitud es rechazada.

## 8. Implementación por capas

El proyecto fue desarrollado utilizando una estructura por capas, lo que permite separar mejor las responsabilidades dentro de cada microservicio. Esto ayuda al trabajo y a que el código sea más ordenado, fácil de revisar y más simple de modificar si en el futuro se necesita agregar o cambiar alguna función.

En la mayoría de los microservicios se utilizó la misma estructura:

**Model:** representa la entidad principal del microservicio y define los datos que serán guardados en la base de datos. Por ejemplo, en el microservicio de profesores se define la clase Profesor, donde se guardan datos como nombre, apellido, correo y teléfono.

**Repository:** se encarga de la comunicación directa con la base de datos. Esta capa permite guardar, buscar, listar, actualizar o eliminar registros sin tener que escribir consultas SQL manualmente para cada operación.

**Service:** contiene la lógica principal del microservicio. Aquí se crean los métodos que permiten realizar las operaciones necesarias, como mostrar todos los registros, buscar por ID, guardar un nuevo dato, actualizarlo o eliminarlo.

**Controller:** es la capa que recibe las solicitudes desde Postman, Swagger o el API Gateway. En esta capa se definen las rutas REST, como GET, POST, PUT y DELETE, para que el usuario pueda interactuar con el sistema.

Evidencias de Profesor:



```

12 @Tag(name = "Profesores", description = "Operaciones relacionadas con los profesores")
13 @RestController
14 @CrossOrigin(origins = "**")
15 @RequestMapping("/profesores")
16 public class ProfesorController
17 {
18     @Autowired Convert @Autowired field into Constructor Parameter
19     private ProfesorService profesorService;
20
21     @Operation(summary = "Mostrar todos los profesores", description = "Obtiene todos los profesores registrados")
22     @GetMapping
23     public List<Profesor> listar() {
24         return profesorService.listarTodos();
25     }
26
27     @Operation(summary = "Buscar profesor por ID", description = "Obtiene un profesor especifico por la id")
28     @GetMapping("/{id}")
29     public ResponseEntity<Profesor> obtener(@PathVariable Long id) {
30         return profesorService.buscarPorId(id)
31             .map(ResponseEntity::ok)
32             .orElse(ResponseEntity.notFound().build());
33     }
34
35     @Operation(summary = "Buscar profesor por nombre", description = "Busca un profesor por su nombre")
36     @GetMapping("/nombre/{nombre}")
37     public List<Profesor> buscarPorNombre(@PathVariable String nombre) {
38         return profesorService.buscarPorNombre(nombre);
39     }
40
41     @Operation(summary = "Buscar profesores por apellido", description = "Busca un profesor por su apellido")
42     @GetMapping("/apellido/{apellido}")
43     public List<Profesor> buscarPorApellido(@PathVariable String apellido) {
44         return profesorService.buscarPorApellido(apellido);
45     }
46
47     @Operation(summary = "Buscar profesor por correo", description = "Busca un profesor por su correo electrónico")
48     @GetMapping("/correo")
49     public ResponseEntity<Profesor> buscarPorCorreo(@RequestParam String correo) {
50         Profesor profesor = profesorService.buscarPorCorreo(correo);
51
52         if (profesor == null) {
53             return ResponseEntity.notFound().build();
54         }
55
56         return ResponseEntity.ok(profesor);
57     }

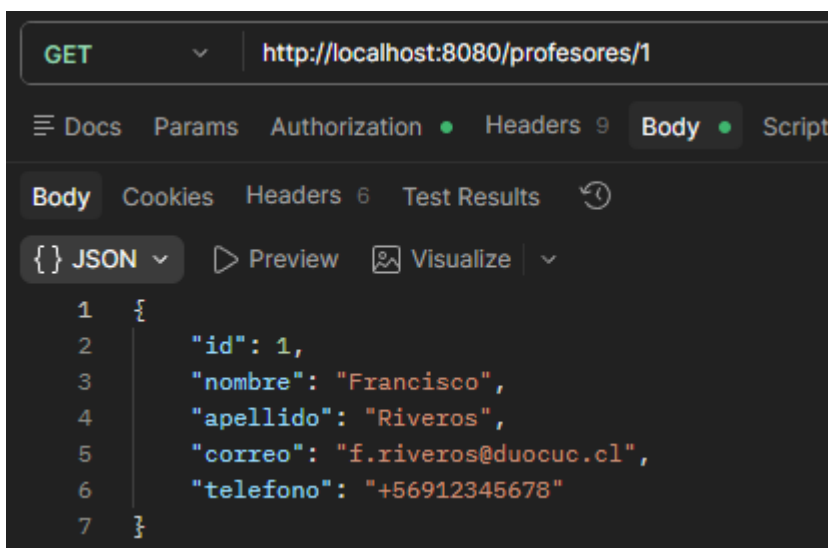
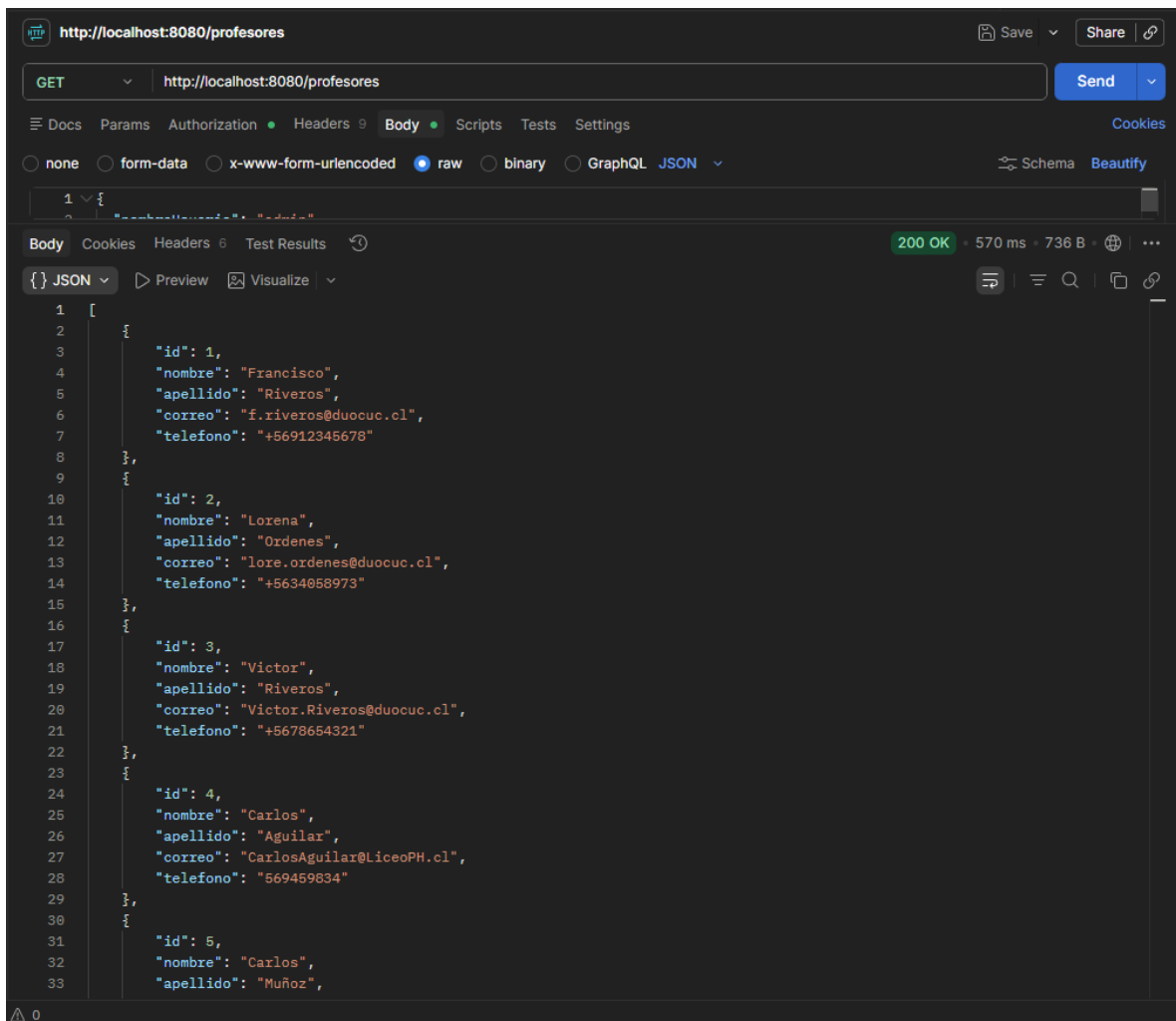
```

```

59     @Operation(summary = "Crear profesor", description = "Crea un nuevo profesor")
60     @PostMapping
61     public ResponseEntity<Profesor> crear(@RequestBody Profesor profesor) {
62         return ResponseEntity.ok(profesorService.guardar(profesor));
63     }
64     @Operation(summary = "Actualizar profesor", description = "Actualiza los datos de un profesor existente")
65     @PutMapping("/{id}")
66     public ResponseEntity<Profesor> actualizar(@PathVariable Long id, @RequestBody Profesor profesor) {
67         return ResponseEntity.ok(profesorService.actualizar(id, profesor));
68     }
69
70     @Operation(summary = "Eliminar profesor", description = "Elimina un profesor")
71     @DeleteMapping("/{id}")
72     public ResponseEntity<Void> eliminar(@PathVariable Long id) {
73         profesorService.eliminar(id);
74         return ResponseEntity.noContent().build();
75     }
76 }

```

## Evidencia en Postman:



PUT ▼ http://localhost:8080/profesores/1

Docs Params Authorization Headers 9 Body Scripts

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw

```
1 {
2   "nombre": "Francisco",
3   "apellido": "Riveros",
4   "correo": "fr.riveros@duocuc.cl",
5   "telefono": "666666666"
6 }
```

Body Cookies Headers 6 Test Results

{ } JSON ▼ Preview Visualize ▼

```
1 {
2   "id": 1,
3   "nombre": "Francisco",
4   "apellido": "Riveros",
5   "correo": "fr.riveros@duocuc.cl",
6   "telefono": "666666666"
7 }
```

**POST** ⌵ <http://localhost:8080/profesores>

☰ Docs Params Authorization ● Headers 9 **Body** ● Scripts

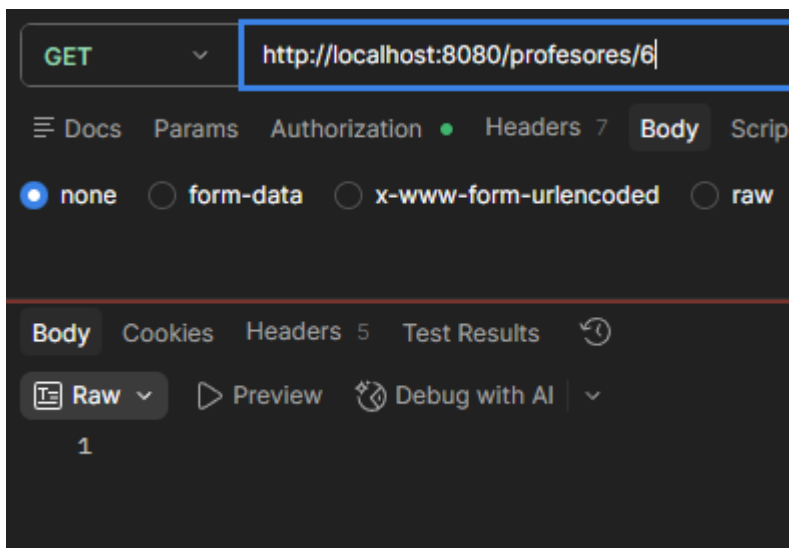
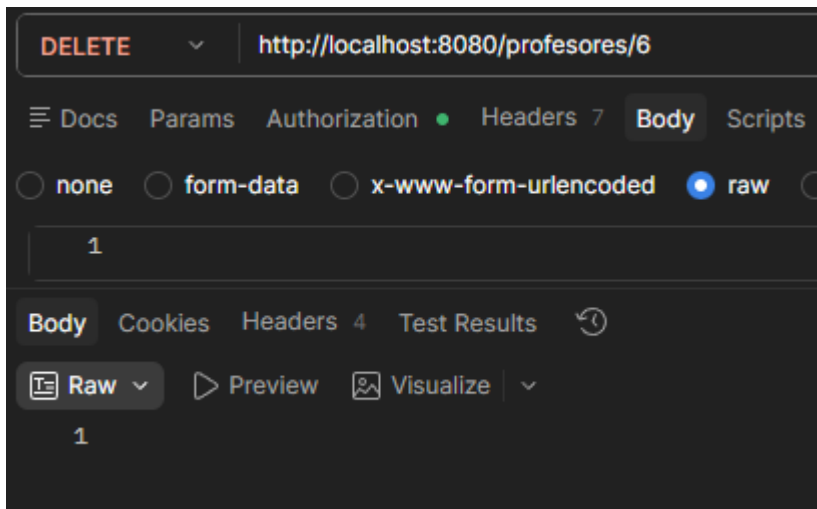
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ b

```
1  {
2    "nombre": "Federico",
3    "apellido": "Valverde",
4    "correo": "fedexvalverde@duocuc.cl",
5    "telefono": "777777777"
6  }
```

**Body** Cookies Headers 6 Test Results ↺

**{ } JSON** ⌵ ▶ Preview 🖼 Visualize ⌵

```
1  {
2    "id": 6,
3    "nombre": "Federico",
4    "apellido": "Valverde",
5    "correo": "fedexvalverde@duocuc.cl",
6    "telefono": "777777777"
7  }
```



## 9. Comunicación entre microservicios

En este proyecto la comunicación entre microservicios se ve principalmente en service-impresiones, ya que este microservicio no trabaja completamente solo. Cuando se registra una impresión, se guardan datos propios de la solicitud, como la cantidad de copias, el estado, la fecha y las notas adicionales. También se guardan los IDs del profesor, curso y asignatura relacionados con esa impresión.

El motivo de guardar estos IDs es que la información completa no se encuentra dentro del mismo microservicio. Por ejemplo, los datos del profesor están en service-profesores, los datos del curso están en service-cursos y los datos de la asignatura están en service-asignaturas. Por eso, cuando se consulta una impresión, service-impresiones utiliza esos IDs para solicitar información a los otros microservicios.

Un ejemplo sería una impresión que tiene:

profesor\_id = 1

curso\_id = 2

Y asignatura\_id = 3.

El microservicio de impresiones guarda esos valores, pero para mostrar el detalle completo consulta a los otros servicios y obtiene datos como el nombre del profesor, su correo, el nombre del curso y la asignatura correspondiente.

Gracias a esto, la respuesta de una impresión no queda limitada solo a números o identificadores. El sistema puede mostrar una información más útil para la imprenta, ya que permite saber quién solicitó la impresión, para qué curso corresponde y a qué asignatura pertenece.

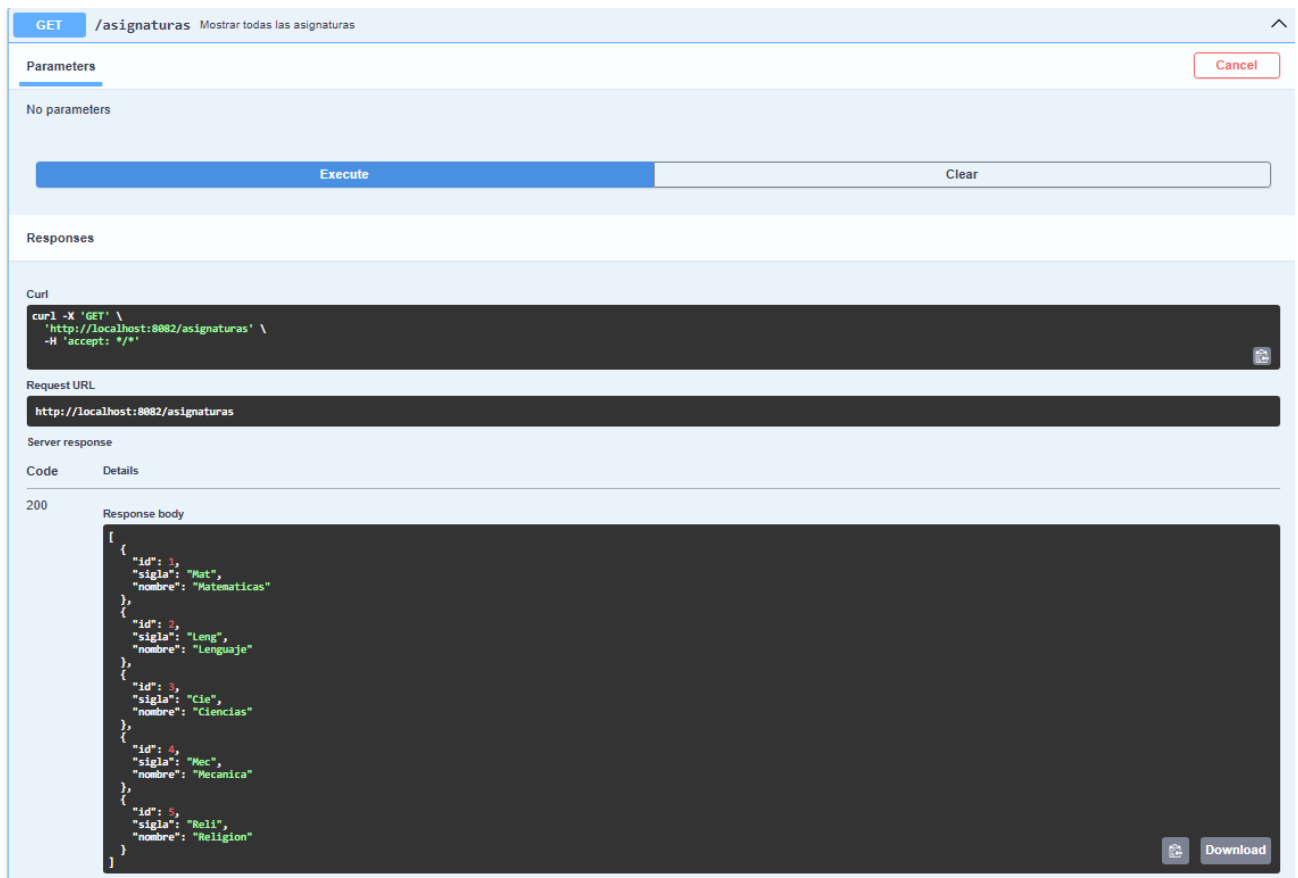
Esta comunicación ayuda a que cada microservicio mantenga su propia responsabilidad. Profesores administra profesores, cursos administra cursos, asignaturas administra asignaturas y por último impresiones administra las solicitudes de impresión. Aun así, cuando el sistema necesita mostrar una solicitud completa, los servicios se comunican entre sí para entregar una respuesta más clara.

## 10. Documentación Swagger

Para documentar los endpoints del sistema se utilizó Swagger. Esta herramienta permite visualizar las rutas disponibles de cada microservicio y probarlas desde el navegador sin tener que escribir manualmente cada URL.

En este proyecto Swagger fue configurado en cada microservicio y también centralizado desde el API Gateway. Esto permite ingresar desde el puerto 8080 y seleccionar el microservicio que se quiere revisar, como profesores, asignaturas, cursos, impresiones, historial, inventario, cola, calidad o retiros.

La documentación muestra los métodos disponibles, como GET, POST, PUT y DELETE. Variando y dependiendo del microservicio. Además de la ruta que se debe utilizar y el tipo de datos que se envía en el Json ya viene estructurado. Esto facilita la revisión del proyecto, ya que permite entender rápidamente que se debe ingresar en cada microservicio. Gracias a Swagger el proyecto queda más ordenado y fácil de revisar, y usar. Ya que se puede acceder a la documentación centralizada desde una sola dirección.



GET en /Asignatura, mostrando todas las asignaturas creadas.

POST

/inventario

Crear nuevo material

⌵

Registra un nuevo material en el inventario

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "nombre": "Caja de corchetes",
  "cantidad": 20,
  "descripcion": "Caja de corchetes"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8086/inventario' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Caja de corchetes",
    "cantidad": 20,
    "descripcion": "Caja de corchetes"
  }'
```

Request URL

```
http://localhost:8086/inventario
```

Server response

| Code | Details                                                                                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><pre>{   "id": 5,   "nombre": "Caja de corchetes",   "cantidad": 20,   "descripcion": "Caja de corchetes" }</pre><div><div>Download</div></div></div> |

POST en /Inventario, mostrando la creación de una caja de corchetes.



POST

/retiros

Crear retiro

^

Crear un nuevo estado de retiro de impresion

Parameters

Cancel

Reset

No parameters

Request body <sup>required</sup>

application/json

{  
 "impresionId": 8,  
 "estadoRetiro": "Lista"  
}

Execute

Clear

Responses

Curl

curl -X 'POST' \  
 'http://localhost:8089/retiros' \  
 -H 'accept: \*/\*' \  
 -H 'Content-Type: application/json' \  
 -d '{  
 "impresionId": 8,  
 "estadoRetiro": "Lista"  
 }'

Request URL

http://localhost:8089/retiros

Server response

Code

Details

200

Response body

{  
 "id": 8,  
 "impresionId": 8,  
 "estadoRetiro": "Lista"  
}

Download

POST en /Retiros, mostrando como asocio a la impresión n8 con estado lista.

```

20     @Operation(summary = "Mostrar todas las asignaturas")
21     @GetMapping
22     public List<Asignatura> listar()
23     {
24         return asignaturaService.listarTodas();
25     }
26
27     @Operation(summary = "Buscar una asignatura por su ID")
28     @GetMapping("/{id}")
29     public ResponseEntity<Asignatura> obtener(@PathVariable Long id)
30     {
31         return asignaturaService.buscarPorId(id)
32             .map(ResponseEntity::ok)
33             .orElse(ResponseEntity.notFound().build());
34     }
35
36     @Operation(summary = "Buscar asignaturas por nombre")
37     @GetMapping("/nombre/{nombre}")
38     public List<Asignatura> buscarPorNombre(@PathVariable String nombre)
39     {
40         return asignaturaService.buscarPorNombre(nombre);
41     }
42
43     @Operation(summary = "Buscar una asignatura por su sigla")
44     @GetMapping("/{sigla/{sigla}")
45     public ResponseEntity<Asignatura> buscarPorSigla(@PathVariable String sigla)
46     {
47         Asignatura asignatura = asignaturaService.buscarPorSigla(sigla);
48         if (asignatura == null)
49         {
50             return ResponseEntity.notFound().build();
51         }
52         return ResponseEntity.ok(asignatura);
53     }
54
55     @Operation(summary = "Crear una nueva asignatura")
56     @PostMapping
57     public ResponseEntity<Asignatura> crear(@RequestBody Asignatura asignatura)
58     {
59         return ResponseEntity.ok(asignaturaService.guardar(asignatura));
60     }
61
62     @Operation(summary = "Actualizar una asignatura existente")
63     @PutMapping("/{id}")
64     public ResponseEntity<Asignatura> actualizar(@PathVariable Long id, @RequestBody Asignatura asignatura)
65     {
66         return ResponseEntity.ok(asignaturaService.actualizar(id, asignatura));
67     }
68
69     @Operation(summary = "Eliminar una asignatura por su id")
70     @DeleteMapping("/{id}")
71     public ResponseEntity<Void> eliminar(@PathVariable Long id)
72     {
73         asignaturaService.eliminar(id);
74         return ResponseEntity.noContent().build();
75     }
76

```

Código de AsignaturaController

```

11 @RequestMapping("/inventario")
12 @CrossOrigin(origins = "**")
13 @Tag(name = "Inventario", description = "Operaciones relacionadas con el inventario de materiales")
14 public class InventarioController
15 {
16     @Autowired Convert @Autowired field into Constructor Parameter
17     private InventarioService inventarioService;
18
19     @Operation(summary = "Mostrar todos los materiales", description = "Muestra todos los materiales disponibles en el inventario")
20     @GetMapping
21     public List<Inventario> listarTodos() {
22         return inventarioService.listarTodos();
23     }
24
25     @Operation(summary = "Buscar materiales por nombre", description = "Busca materiales que contengan el nombre especificado")
26     @GetMapping("/{nombre}/{nombre}")
27     public List<Inventario> buscarPorNombre(@PathVariable String nombre) {
28         return inventarioService.buscarPorNombre(nombre);
29     }
30
31     @Operation(summary = "Crear nuevo material", description = "Registra un nuevo material en el inventario")
32     @PostMapping
33     public Inventario crear(@RequestBody Inventario inventario) {
34         return inventarioService.guardar(inventario);
35     }
36
37
38     @Operation(summary = "Actualizar material existente", description = "Actualiza la información de un material existente en el inventario")
39     @PutMapping("/{id}")
40     public Inventario actualizar(@PathVariable Long id, @RequestBody Inventario inventario) {
41         return inventarioService.actualizar(id, inventario);
42     }
43
44     @Operation(summary = "Eliminar material por id", description = "Elimina un material del inventario por su ID")
45     @DeleteMapping("/{id}")
46     public void eliminar(@PathVariable Long id) {
47         inventarioService.eliminar(id);
48     }
49
50     @Operation(summary = "Agregar x cantidad a material", description = "Agrega una cantidad específica a un material existente en el inventario")
51     @PostMapping("/{id}/agregar")
52     public Inventario agregarCantidad(@PathVariable Long id, @RequestParam Integer cantidad) {
53         return inventarioService.agregarCantidad(id, cantidad);
54     }
55
56
57     @Operation(summary = "Quitar x cantidad de material", description = "Quita una cantidad específica de un material existente en el inventario")
58     @PostMapping("/{id}/quitar")
59     public Inventario quitarCantidad(@PathVariable Long id, @RequestParam Integer cantidad) {
60         return inventarioService.quitarCantidad(id, cantidad);
61     }
62 }

```

Código de InventarioController

```

11 @RestController
12 @RequestMapping("/retiros")
13 @CrossOrigin(origins = "**")
14 @Tag(name = "Retiros", description = "Operaciones relacionadas para saber si una impresion ha sido retirada o no")
15 public class RetiroController {
16
17     @Autowired Convert @Autowired field into Constructor Parameter
18     private RetiroService retiroService;
19
20     @Operation(summary = "Mostrar estado de todos los retiros", description = "Muestra el estado de todas las impresiones")
21     @GetMapping
22     public List<Retiro> listar()
23     {
24         return retiroService.listarTodos();
25     }
26
27     @Operation(summary = "Crear retiro", description = "Crear un nuevo estado de retiro de impresion")
28     @PostMapping
29     public Retiro crear(@RequestBody Retiro retiro)
30     {
31         return retiroService.guardar(retiro);
32     }
33
34     @Operation(summary = "Actualizar retiro", description = "Actualiza el estado de retiro de una impresion por ID")
35     @PutMapping("/{id}")
36     public Retiro actualizar(@PathVariable Long id, @RequestBody Retiro retiro) {
37         return retiroService.actualizar(id, retiro);
38     }
39
40     @Operation(summary = "Eliminar retiro", description = "Elimina un estado de retiro de impresion por ID")
41     @DeleteMapping("/{id}")
42     public void eliminar(@PathVariable Long id) {
43         retiroService.eliminar(id);
44     }
45 }

```

Codigo de RetiroController

## 11. Pruebas unitarias

Para validar el funcionamiento del proyecto se implementaron pruebas unitarias utilizando **JUnit** y **Mockito**. Estas pruebas permiten revisar que los métodos principales de los microservicios funcionen correctamente sin tener que levantar todo el sistema completo ni depender directamente de la base de datos.

Las pruebas unitarias se enfocan en testear el código, principalmente la capa Service, ya que en esta capa se encuentra la lógica de cada microservicio. De esta forma se puede comprobar si los métodos para listar, buscar, guardar, actualizar o eliminar datos entregan el resultado esperado.

JUnit se utilizó para crear y ejecutar las pruebas, mientras que Mockito se utilizó para simular el comportamiento del Repository. Esto permite probar el Service sin depender directamente de la base de datos real. Por ejemplo, en vez de consultar realmente la tabla de profesores, Mockito simula una respuesta del Repository y permite comprobar si el método del Service funciona correctamente.

Las pruebas fueron realizadas siguiendo la estructura AAA:

**Arrange (Preparar):** se preparan los datos necesarios para la prueba, como un curso de ejemplo o una respuesta simulada del Repository.

**Act (Actuar):** se ejecuta el método del service que se quieren probar, por ejemplo, en este caso, **ListarTodos**, **BuscarPorId**, **Guardar** o **Actualizar**

**Assert (Verificar):** se comprueba que el resultado obtenido sea el esperado, como validar que el curso exista, ejecutar operaciones como **FindBy**, **FindAll**. Las pruebas unitarias permiten comprobar que estas operaciones funcionen correctamente sin conectarse a la base de datos.

Estas mismas pruebas se aplicaron sobre los microservicios de: Profesores, asignaturas, cursos, historial, impresiones y inventario.

```
21 @ExtendWith(MockitoExtension.class)
22 class ServiceProfesoresApplicationTests {
23
24     @Mock
25     private ProfesorRepository profesorRepository;
26
27     @InjectMocks
28     private ProfesorService profesorService;
29
30     @Test
31     void listarTodosTest() {
32         Profesor profesor1 = new Profesor(id: 1L, nombre: "Francisco", apellido: "Riveros", correo: "f.riveros@duocuc.cl", telefono: "912345678");
33         Profesor profesor2 = new Profesor(id: 2L, nombre: "Carlos", apellido: "Muñoz", correo: "c.munoz@duocuc.cl", telefono: "987654321");
34
35         when(profesorRepository.findAll()).thenReturn(Arrays.asList(profesor1, profesor2));
36
37         List<Profesor> resultado = profesorService.listarTodos();
38
39         assertEquals(expected: 2, resultado.size());
40         assertEquals(expected: "Francisco", resultado.get(index: 0).getNombre());
41
42         verify(profesorRepository, times(wantedNumberOfInvocations: 1)).findAll();
43     }
44
45     @Test
46     void buscarPorIdTest() {
47         Profesor profesor = new Profesor(id: 1L, nombre: "Francisco", apellido: "Riveros", correo: "f.riveros@duocuc.cl", telefono: "912345678");
48     }
49 }
```

PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ServiceProfesoresApplicationTests \$(symbol-namespace) com.imprenta.service\_profesores + \$(project) service-profesores %TESTE 6,guardarTest(com.imprenta.ser
- actualizarTest() \$(symbol-class) ServiceProfesoresApplicationTests + \$(symbol-namespace) com.imprenta.service\_profesores + \$(project) service-prof... %TESTS 7,buscarPorIdTest(com.imprenta
- buscarPorIdTest() \$(symbol-class) ServiceProfesoresApplicationTests + \$(symbol-namespace) com.imprenta.service\_profesores + \$(project) service-p... %TESTE 7,buscarPorIdTest(com.imprenta
- eliminarTest() \$(symbol-class) ServiceProfesoresApplicationTests + \$(symbol-namespace) com.imprenta.service\_profesores + \$(project) service-profe... %RUNTIME1160
- guardarTest() \$(symbol-class) ServiceProfesoresApplicationTests + \$(symbol-namespace) com.imprenta.service\_profesores + \$(project) service-profe...
- listarTodosTest() \$(symbol-class) ServiceProfesoresApplicationTests + \$(symbol-namespace) com.imprenta.service\_profesores + \$(project) service-pr...

> 21 older results

Aquí se pueden ver las pruebas unitarias realizadas en el microservicio **service-profesores**. Se utilizó Mockito para simular el comportamiento de **ProfesorRepository** y así probar la capa **Service** sin conectarse directamente a la base de datos.

También se validan los métodos `listarTodos`, `buscarPorId`, `guardar`.

```
15 @ExtendWith(MockitoExtension.class)
16 class AsignaturaServiceTest {
17
18     @Mock
19     private AsignaturaRepository asignaturaRepository;
20
21     @InjectMocks
22     private AsignaturaService asignaturaService;
23
24     @Test
25     void listarTodasTest() {
26         Asignatura asignatura1 = new Asignatura(id: 1L, sigla: "MAT", nombre: "Matematicas");
27         Asignatura asignatura2 = new Asignatura(id: 2L, sigla: "LEN", nombre: "Lenguaje");
28
29         when(asignaturaRepository.findAll()).thenReturn(Arrays.asList(asignatura1, asignatura2));
30
31         List<Asignatura> resultado = asignaturaService.listarTodas();
32
33         assertEquals(expected: 2, resultado.size());
34         assertEquals(expected: "Matematicas", resultado.get(index: 0).getNombre());
35
36         verify(asignaturaRepository, times(wantedNumberOfInvocations: 1)).findAll();
37     }
38
39     @Test
40     void buscarPorIdTest() {
41         Asignatura asignatura = new Asignatura(id: 1L, sigla: "MAT", nombre: "Matematicas");
42
43         when(asignaturaRepository.findById(id: 1L)).thenReturn(Optional.of(asignatura));
44
45         Optional<Asignatura> resultado = asignaturaService.buscarPorId(id: 1L);
46
47         assertTrue(resultado.isPresent());
48         assertEquals(expected: "MAT", resultado.get().getSigla());
49
50         verify(asignaturaRepository, times(wantedNumberOfInvocations: 1)).findById(id: 1L);
51     }
52
53     @Test
54     void buscarPorNombreTest() {
55
56     }
57
58     @Test
59     void buscarPorSiglaTest() {
60
61     }
62
63     @Test
64     void eliminarTest() {
65
66     }
67
68     @Test
69     void guardarTest() {
70
71     }
72
73     @Test
74     void actualizarTest() {
75
76     }
77 }
```

PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ AsignaturaServiceTest \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-asignaturas
- ✓ actualizarTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-asignat...
- ✓ buscarPorIdTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-asign...
- ✓ buscarPorNombreTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) servic...
- ✓ buscarPorSiglaTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-as...
- ✓ eliminarTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-asignaturas
- ✓ guardarTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-asignaturas
- ✓ listarTodasTest() \$(symbol-class) AsignaturaServiceTest < \$(symbol-namespace) com.imprenta.service\_asignaturas.service < \$(project) service-asigna...

> 22 older results

Aquí se pueden ver las pruebas unitarias para el microservicio de asignaturas. Se utilizó Mockito para simular el comportamiento de **AsignaturaRepository** y así probar Service sin conectarse directamente a la base de datos.

también se validan los métodos listarTodas, buscarPorId, buscarPorNombre, buscarPorSigla, guardar, actualizar y eliminar.

```
16 @ExtendWith(MockitoExtension.class)
17 class CursoServiceTest
18 {
19
20     @Mock
21     private CursoRepository cursoRepository;
22
23     @InjectMocks
24     private CursoService cursoService;
25
26     @Test
27     void listarTodosTest()
28     {
29         Curso curso1 = new Curso(id: 1L, nombre: "1 Basico A", nivel: "Basico", jornada: "Diurna");
30         Curso curso2 = new Curso(id: 2L, nombre: "1 Medio A", nivel: "Medio", jornada: "Nocturno");
31
32         when(cursoRepository.findAll()).thenReturn(Arrays.asList(curso1, curso2));
33
34         List<Curso> resultado = cursoService.listarTodos();
35
36         assertEquals(expected: 2, resultado.size());
37         assertEquals(expected: "1 Basico A", resultado.get(index: 0).getNombre());
38
39         verify(cursoRepository, times(wantedNumberOfInvocations: 1)).findAll();
40     }
41
42     @Test
43     void buscarPorIdTest()
44     {
45         Curso curso = new Curso(id: 1L, nombre: "1 Basico A", nivel: "Basico", jornada: "Diurna");
46
47         when(cursoRepository.findById(id: 1L)).thenReturn(Optional.of(curso));
48
49         Optional<Curso> resultado = cursoService.buscarPorId(id: 1L);
50
51         assertTrue(resultado.isPresent());
52         assertEquals(expected: "1 Basico A", resultado.get().getNombre());
53
54         verify(cursoRepository, times(wantedNumberOfInvocations: 1)).findById(id: 1L);
55     }
56
57     @Test
58     void guardarTest()
59     {
60
61     }
62 }
```

PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ CursoServiceTest \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ actualizarTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ buscarPorIdTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ buscarPorJornadaTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ buscarPorNivelTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ buscarPorNombreTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ buscarPorNombreYJornadaTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ eliminarTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ guardarTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos
- ✓ listarTodosTest() \$(symbol-class) CursoServiceTest < \$(symbol-namespace) com.imprenta.service\_cursos.Service < \$(project) service-cursos

> 23 older results

main 0L 11 0 0 40 1 1 Spring Boot-ServiceProfesoresApplication <service-profesores> (imprenta-Riveros-Vargas-Diaz-002D) Java: Ready

Aquí se pueden ver las pruebas unitarias realizadas en el microservicio **cursos**. Se utilizó Mockito para simular el comportamiento de **CursoRepository** y así probar la capa **Service** sin conectarse directamente a la base de datos.

También se validan los métodos `listarTodos`, `buscarPorId`, `buscarPorNombre`, `buscarPorNivel`, `buscarPorJornada`, `guardar`, `actualizar` y `eliminar`



```
16 @ExtendWith(MockitoExtension.class)
17 class HistorialServiceTest {
18
19     @Mock
20     private HistorialRepository historialRepository;
21
22     @InjectMocks
23     private HistorialService historialService;
24
25     @Test
26     void listarTodosTest() {
27         Historial historial1 = new Historial(id: 1L, impresionId: 10L, accion: "CREADA", LocalDateTime.now());
28         Historial historial2 = new Historial(id: 2L, impresionId: 11L, accion: "CREADA", LocalDateTime.now());
29
30         when(historialRepository.findAll()).thenReturn(Arrays.asList(historial1, historial2));
31
32         List<Historial> resultado = historialService.listarTodos();
33
34         assertEquals(expected: 2, resultado.size());
35         assertEquals(expected: 10L, resultado.get(index: 0).getImpresionId());
36
37         verify(historialRepository, times(wantedNumberOfInvocations: 1)).findAll();
38     }
39
40     @Test
41     void buscarPorImpresionTest() {
42         Historial historial = new Historial(id: 1L, impresionId: 10L, accion: "CREADA", LocalDateTime.now());
43
44         when(historialRepository.findByImpresionId(impresionId: 10L)).thenReturn(Arrays.asList(historial));
45
46         List<Historial> resultado = historialService.buscarPorImpresion(impresionId: 10L);
47
48         assertEquals(expected: 1, resultado.size());
49         assertEquals(expected: "CREADA", resultado.get(index: 0).getAccion());
50
51         verify(historialRepository, times(wantedNumberOfInvocations: 1)).findByImpresionId(impresionId: 10L);
52     }
53
54     @Test
55     void guardarTest() {
56         Historial historial = new Historial(id: null, impresionId: 10L, accion: "CREADA", fecha: null);
57         Historial historialGuardado = new Historial(id: 1L, impresionId: 10L, accion: "CREADA", LocalDateTime.now());
58     }
59 }
```

PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- HistorialServiceTest \$(symbol-namespace) com.imprenta.service\_historial.Service - \$(project) service-historial
- guardarTest() \$(symbol-class) HistorialServiceTest - \$(symbol-namespace) com.imprenta.service\_historial.Service - \$(project) service-historial

> 24 older results

Aquí se pueden ver las pruebas unitarias realizadas en el microservicio service-historial. Se utilizó Mockito para simular el comportamiento de HistorialRepository y así probar la capa **Service** sin conectarse directamente a la base de datos

También se validan los métodos listarTodos, buscarPorImpresion y guardar.

```
19 class ImpresionServiceTest
20
21 @Mock
22 private WebClient.Builder webClientBuilder;
23
24 @InjectMocks
25 private ImpresionService impresionService;
26
27 @Test
28 void listarTodasTest()
29 {
30     Impresion impresion1 = new Impresion();
31     impresion1.setId(id: 1L);
32     impresion1.setProfesorId(profesorId: 1L);
33     impresion1.setAsignaturaId(asignaturaId: 1L);
34     impresion1.setCursoId(cursoId: 1L);
35     impresion1.setCantidadCopias(cantidadCopias: 50);
36     impresion1.setEstado(estado: "PENDIENTE");
37     impresion1.setFechaSolicitud(LocalDate.now());
38     impresion1.setNotasAdicionales(notasAdicionales: "Prueba 1");
39
40     Impresion impresion2 = new Impresion();
41     impresion2.setId(id: 2L);
42     impresion2.setProfesorId(profesorId: 1L);
43     impresion2.setAsignaturaId(asignaturaId: 1L);
44     impresion2.setCursoId(cursoId: 1L);
45     impresion2.setCantidadCopias(cantidadCopias: 30);
46     impresion2.setEstado(estado: "LISTO");
47     impresion2.setFechaSolicitud(LocalDate.now());
48     impresion2.setNotasAdicionales(notasAdicionales: "Prueba 2");
49
50     when(impresionRepository.findAll()).thenReturn(Arrays.asList(impresion1, impresion2));
51
52     List<Impresion> resultado = impresionService.listarTodas();
53
54     assertEquals(expected: 2, resultado.size());
55     assertEquals(expected: "PENDIENTE", resultado.get(index: 0).getEstado());
56
57     verify(impresionRepository, times(wantedNumberOfInvocations: 1)).findAll();
58 }
59
60 @Test
61 void buscarPorIdTest()
```

PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ImpresionServiceTest \$(symbol-namespace) com.imprenta.service\_impresiones.service • \$(project) service-impresiones
- buscarPorIdTest() \$(symbol-class) ImpresionServiceTest • \$(symbol-namespace) com.imprenta.service\_impresiones.service • \$(project) service-impr...

> 25 older results

Aquí se pueden ver las pruebas unitarias realizadas en el microservicio **service-impresiones**. Se utilizó Mockito para simular el comportamiento de **ImpresionRepository** y así probar la capa **Service** sin conectarse directamente a la base de datos.

También se validan los métodos `listarTodos` y `buscarPorId`.

```
16 class InventarioServiceTest
17 {
18
19     @Mock
20     private InventarioRepository inventarioRepository;
21
22     @InjectMocks
23     private InventarioService inventarioService;
24
25     @Test
26     void listarTodosTest()
27     {
28         Inventario item1 = new Inventario(id: 1L, nombre: "Resma carta", cantidad: 20, descripcion: "Papel blanco tamaño carta");
29         Inventario item2 = new Inventario(id: 2L, nombre: "Tóner negro", cantidad: 5, descripcion: "Tóner para impresora principal");
30
31         when(inventarioRepository.findAll()).thenReturn(Arrays.asList(item1, item2));
32
33         List<Inventario> resultado = inventarioService.listarTodos();
34
35         assertEquals(2, resultado.size());
36         assertEquals("Resma carta", resultado.get(index: 0).getNombre());
37
38         verify(inventarioRepository, times(1)).findAll();
39     }
40
41     @Test
42     void buscarPorNombreTest()
43     {
44         Inventario item = new Inventario(id: 1L, nombre: "Resma carta", cantidad: 20, descripcion: "Papel blanco tamaño carta");
45
46         when(inventarioRepository.findByNameContainingIgnoreCase(nombre: "resma"))
47             .thenReturn(Arrays.asList(item));
48
49         List<Inventario> resultado = inventarioService.buscarPorNombre(nombre: "resma");
50
51         assertEquals(1, resultado.size());
52         assertEquals("Resma carta", resultado.get(index: 0).getNombre());
53
54         verify(inventarioRepository, times(1)).findByNameContainingIgnoreCase(nombre: "resma");
55     }
56
57     @Test
58     void guardarTest()
59     {
60         Inventario item = new Inventario(id: null, nombre: "Espirales", cantidad: 100, descripcion: "Espirales para anillado");
61     }
62 }
```

PROBLEMS 41 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ InventarioServiceTest \$(symbol-namespace) com.imprenta.service\_inventario.service + \$(project) service-inventario
- ✓ guardarTest() \$(symbol-class) InventarioServiceTest + \$(symbol-namespace) com.imprenta.service\_inventario.service + \$(project) service-inventario

> 26 older results

%TESTC 1 v2  
%TSTTREE2,com.imprenta.serv  
ce.InventarioServiceTest]  
%TSTTREE3,guardarTest(com.  
.service.InventarioService  
%TESTS 3,guardarTest(com.  
%TESTE 3,guardarTest(com.

Aquí se pueden ver las pruebas unitarias realizadas en el microservicio **service-inventario**. Se utilizó Mockito para simular el comportamiento de **InventarioRepository** y así probar la capa **Service** sin conectarse directamente a la base de datos.

También se validan los métodos listarTodos, buscarPorNombre, guardar, actualizar, eliminar, agregarCantidad y quitarCantidad.